SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	V. Yeshwanth	**Reg. No.:**	192511300
Section / Batch:		**Date:**	02/09/2026

Code of Conduct

I V. Yeshwanth (192511300) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: V.Y

Instructions

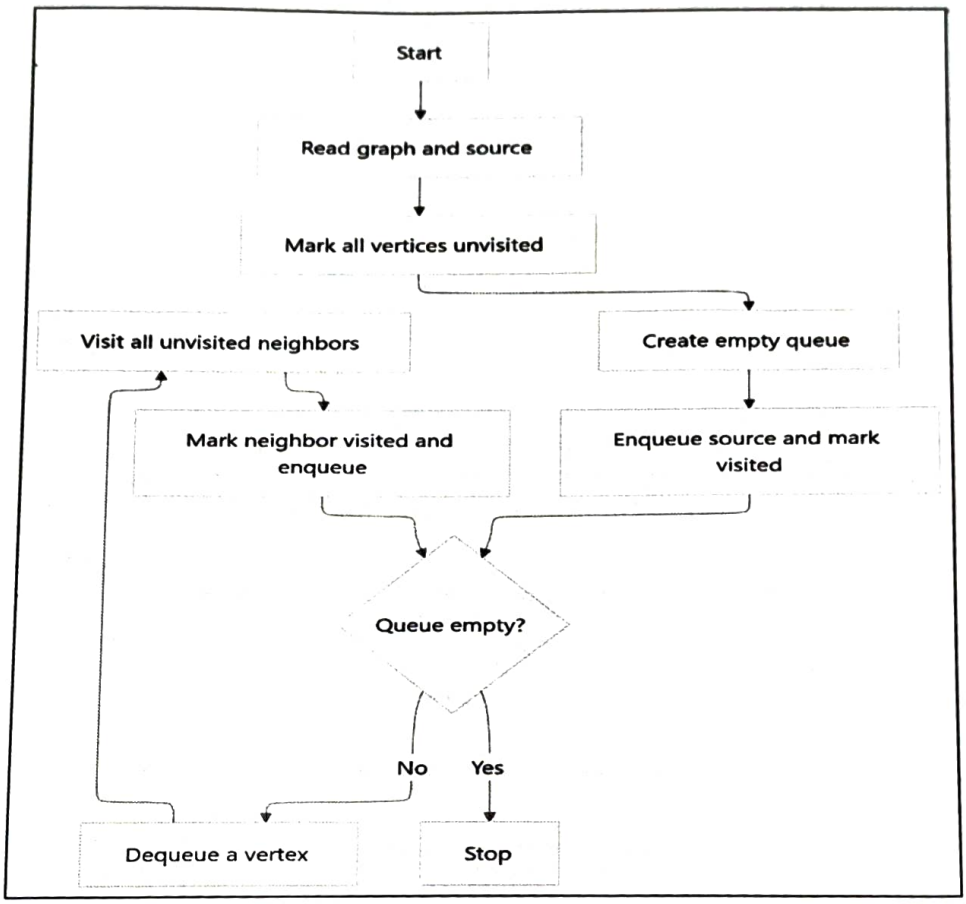
- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Flowchart-To-Code Conversion

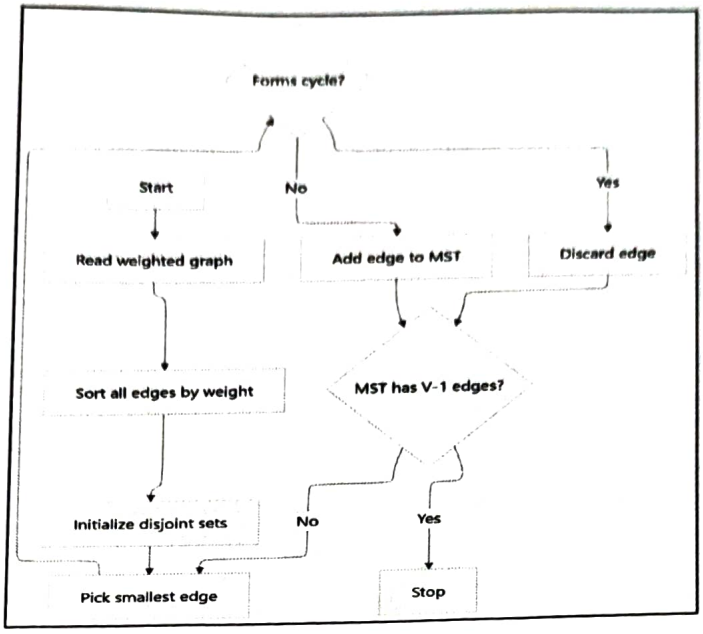
1. Convert the flowchart for Breadth First Search traversal into code. **(20 Marks)**

Flowchart Diagram:



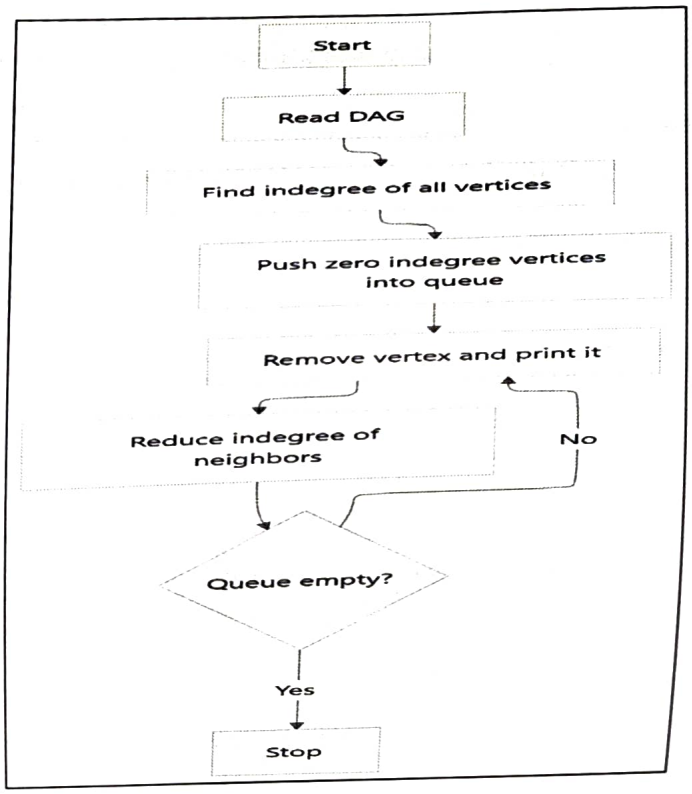
2. Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



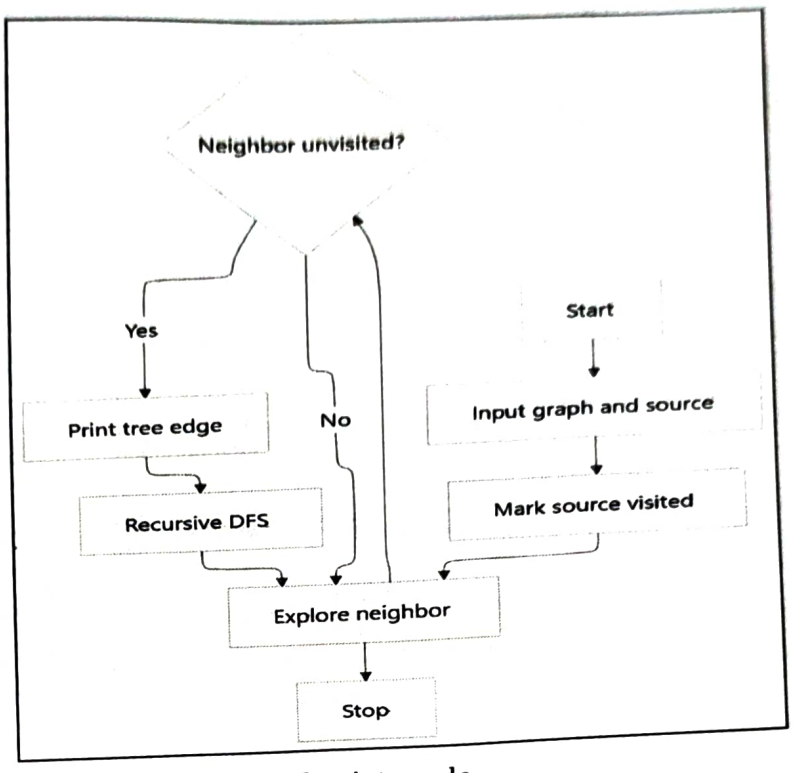
3. Convert the flowchart for topological sort into code. **(20 Marks)**

Flowchart Diagram:



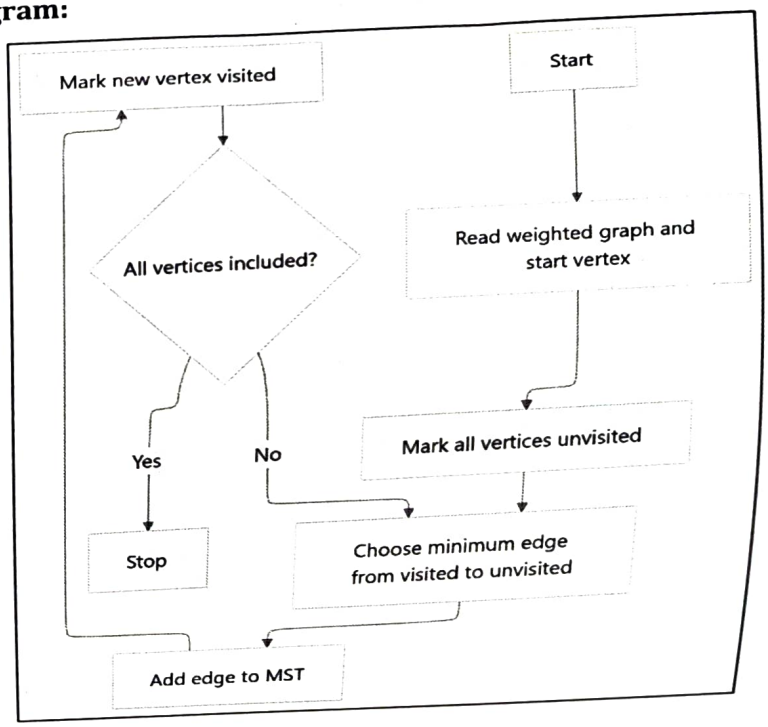
4. Convert the flowchart for generating a DFS tree into code. *(20 Marks)*

Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code. **(20 Marks)**

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

```
#include<stdio.h>
int main()
{
  int graph[10][10];
  int visited[10] = {0};
  int queue[10];
  int n, source;
  int front = 0, rear = 0;
  int i, j, vertex;
  Print("Enter number of vertices:");
  Scant("%d", &n)
  Print("Enter the adjacency mattix; \n");
  for(i=0; i<n; i++)
  { for(j=0; j<n; j++)
    {
  }print("Enter Source vertex; ");
  scant("%d", &source);
queue[rear] = source.
rear++;
visited[source] = 1;
Print f("BFS traversal;");
 While(front<rear)
{ vertex = queue(front);
for(i=0; i<n; i++)
{
  If(graph{vertex][i] = 188
      visited[i] = 0]
     visited[i] = 1;
     queue[rear) = i;
   } rear;
 } }
```

return 0
}

INPUT:

Enter number of vertices: 5
Enter the adjacency matix:

0 1 1 0 0
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
0 0 1 1 0

Enter source vertex: 0

output

BFS Traversal: 0, 1, 2, 3, 4

2.

```
# include <stdio.h>
struct stage
};
    int u, v, weight;
}

int parent [20];
int find (int i)
{
  while (parent (i) != i)
      i = parent [i];
  return i;
}
```

```
void unionset(int u, int v)
{
int root u = find (u);
int root v = find (v);
parent [root v] = 100+u;
}
int main ()
{
  struct tage edge 150), temp);
  Int v, t;
  int i, j;
  Int count = 0;
  int cotal cost = 0;
Print ("enter number of vertices: ");
Scant ("%d", &v);
Printf ("enter number of edges: ");
Scant ("%d", &t);
Print ("tnter edges (source destination
      weight ): in");
for (i=0; i<t; i++)
{
   Scant ("obd %od %od"
         edges (i). u;
         edges (i). v;
         edges (i). weight);
```

```
    }

  for (i=0; i<e-1; i++)
  {
    for
      if(edges(j).weight >edges[j+1]weight)
    {
      temp = edges [ij;
      edges [ij] = edges [j+1];
      edges[j+1] = temp;
      }

    }
}

for
    (i=0; i<v; i++)
{ Parent (j) = j;
}
Printf (" In tages in minimum spanning
        free: In");
to (i=0; i<t d count <v-1; i++)
{
    int u  = edges[ij.u;
    int v = edges [ij.v;
    if (find (u) = find (v))
      Print ("%d ..... %od = %. In."
                u, v, edges [i].weight];
```

```
    cost = totalcost + edges[i].weight;
    unionset(u,v);
    count++;
  }
else
{
  printf("%d--%d=%d dislarded.
          (cycle)\n", u, v, edges[i].weight);
  }
}
  printf("\n minimum cost = %d\n", totalcost);
  return 0;
}
```

Input:-

Enter number of vertices = 4

Enter number of edges = 5

Enter edges(source destination weight).

0	1	10
0	2	6
0	3	5
1	3	15
2	3	4

output

tages in minimum spanning Tree.

2 - - 3 = 4
0 - - 3 = 5 disorder (cycle)
0 - - 2 = 6

- - - 1 = 10

minimum cost = 19.

5.

```
#include<stdio.h>
int main()
{ int a[10][10], indegree[10]={0},
int queue[10], front=0 rear=0";
int n,i,j,v, count=0
Print("enter number of vertices:");
Scant("%d", &n);
Printf("enter adjacency matrix:\n");
for(i=0; i<n; i++)
for(j=0; j<n; j++)
  Scant("%d", &a[i][j]);
For[i=0; i<n; i++]
for(j=0; j<n; i++)
  if(a[i][i]==1)
    indegree[i]++;
for(i=0; 1<n; 1++)
  if(indegree(i)==0)
  queue['rear++]=1,
Print("Topological Sort");
while(front<rear)
{ v=queue(front++);
  Print("%d", v);
  count++;
```

```
for (i=0; i<n; i++)
  if (a[v][i]==1)
  {
    indegree[i]--;
  if (indegree[i]==0)
      queue[rear++]=i;
  }
}
}
if (count!=n)
    printf("\n cycle exists");
    return 0;
}
```

INPUT:

Enter number of vertices: 6

Enter adjacency matrix

0	1	1	0	0	0
0	0	0	1	0	0
0	0	0			0
0	0	0	1	1	
0	0	0	0	0	1
0	0	0	0	0	0

OUTPUT

Topological sort: 0 1 2 3 4 5

DFS Tree

```
# include <stdio.h>
int graph[10][10], visited[10]
int n;
void DFS(int v)
{
  int i;
for (i=0; i<n; i++)
{
  if (graph[v][i]==1)
  {
    if (visited[i]==0)
{
printf ("%d %d -> %d\n", v, i)
visiter[i]=1;
DFS(i);
}
}
}

nt main()
{
  int source (e, i, j);
Printf . p" Enter number of vertices%y,
Scant ("%d", &n);
$rint f ("enter adiacency matrix \n");
```

```
for (i=0; i<n; i++)
for (i=0; i<n; j++)
  scanf ("%d", graph[i][i]);
Printf ("Enter source vertex:");
scanf ("%d", source)
visited (source) = 1;
Print("In DFS Tree edges: \n");
return 0;
```

Input.

Enter number of vertices: 5

Enter adjacency matrix:

0 1 0 0

1 0 0 1 0

1 0 0 0 1

0 1 0 0 1

0 0 1 1 0

Enter source vertex: 0

Output

DFS Tree edges:

0 → 1

1 → 3

3 → 4

4 → 2

```
# include <stdio.h>

int main()
{
int a[10][10], v[10]={0};
int n,i,j,k,min,x,y,cost=0
printf("Enter number of vertices \n");
scanf("%d",&n);
Print+("Enter weighted adjacency matrix \n
        \n");
for(i=0; i<n; i++)
for(j=0; j<n; j++)
v[0]=1
Print("MST edges:\n");
for(k=0; k<n-1; k++)
{ min=9999;
for(i=0; i<n; i++)
if(v[i])
for(j=0; j<n; j++)
if(!v[j] && a[i][j] && a[i][j]<min)
{
min=a[i][j];
x=i;
y=j
```

```
int ("%d %d - %d %d = %d\n", x, y, min);
cost += min;
v[y] = 1;
}
printf("\nMinimum cost = %d", cost);
return 0;
}
```

Input.

```
4
0   10   6   5
10  0    0   15
0   0    0   4
6   0    0   0
5   15   6   0
```

OUTPUT:

MST tages

0 - 3 = 5

3 - 2 = 4

0 - 1 = 10

minimum cost = 19.